

Reviewer Pack — Tusnady 2-Box Discrepancy of Clause-Polarity Cloud Bounds DP...

Entry d8440effcf4b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 17:45:54 UTC

Tusnady 2-Box Discrepancy of Clause-Polarity Cloud Bounds DPLL Size

Entry ID: d8440effcf4b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 17:45:54 UTC

1. Conjecture

Field A (mathematical branch): Geometric combinatorial discrepancy — Tusnady’s problem and the Chazelle–Matousek axis-aligned 2-box range discrepancy of finite point sets in $\mathbb{R}^d$ (Beck 1981; Roth 1954; Schmidt 1972; Matousek ‘Geometric Discrepancy’ 1999; Chazelle ‘The Discrepancy Method’ 2000). The 2-marginal box discrepancy is computed by enumerating coordinate pairs and sign quadrants — an $O(n^2 m)$ combinatorial functional using only $|\cdot|$ and $\max$ (non-ring; no $\mathbb{F}_q$ polynomial extension exists since $\max$ -of-integer-deviations is not preserved under polynomial ring lifts), so the bridge is Aaronson–Wigderson algebrization-safe. An arXiv search ‘Tusnady discrepancy’ AND (‘DPLL’ OR ‘tree-Resolution’ OR ‘CNF refutation’ OR ‘proof complexity’) returns 0 direct hits and <5 adjacent papers (Chen–Matousek–Skraganov on Tusnady–Beck lower bounds, never on CNF refutations). Distinct from blacklisted Bourgain–Tzafriri sub-column dispersion (column-subset operator norms on sign matrices), Halasz L^2 spectrum discrepancy (Laplacian-spectrum CDF deviation), Beck-style signed-support L_∞ discrepancy on incidence rows, and Mostar/Forman–Ricci structural-graph invariants — none of which is an axis-aligned 2-box range discrepancy on the clause-polarity point cloud. **Field B** (complexity object): Tree-like Resolution / DPLL refutation size $t(F)$ for random unsatisfiable 3-CNFs F at clause density α in $[4.0, 5.0]$ (Chvatal–Szemerédi 1988; Beame–Karp–Pitassi–Saks 2002; Ben-Sasson–Wigderson 2001; Achlioptas–Beame–Molloy 2004), in the Cook–Reckhow–Krajíček bounded-arithmetic regime where $\neg F$ is Σ^b_0 and $\log_2 t(F)$ controls V^0 -witnessing complexity. The invariant $D_2(F)$ is STRUCTURAL on the clause LIST (multiplicities, polarities), so two unsat 3-CNFs with identical satisfiability function (both $\equiv \text{FALSE}$) can have very different $D_2(F)$, shielding the invariant from the Razborov–Rudich natural-proofs barrier.

Statement:

For an unsatisfiable 3-CNF F with m clauses on n variables, define the 2-marginal polarity discrepancy $D_2(F) := \max_{1 \leq i < j \leq n \text{ and } (s,t) \in \{-,+\}^2} |c_{ij}^{st}(F) - \mu_{ij}^{st}|$, where $c_{ij}^{st}(F)$ is the number of clauses of F containing literal x_i^s AND literal x_j^t , and $\mu_{ij}^{st} = m \cdot (3/n) \cdot (2/(n-1)) \cdot (1/4)$ is the expected count under the uniform random 3-CNF model with parameters (m,n) . Conjecture: for unsat 3-CNFs F drawn at clause density $\alpha = 4.5$ on n in $\{12, 14, 16, 18, 20\}$ variables, the Spearman rank correlation between $-D_2(F)/\sqrt{m}$ and $\log_2 t^*(F)$ across 30 seeds $\times$ 5 sizes (150 instances) is $\geq +0.30$ with two-sided $p < 0.05$. Equivalently, formulas whose clause-polarity cloud is more uniformly spread over axis-aligned 2-box ranges tend to have

provably larger DPLL refutation trees, and the inequality is falsified by a single (30 seed $\times$ 5 size) ensemble whose empirical correlation falls below 0.30.

Rationale (proposer’s reasoning):

Geometric discrepancy quantifies how evenly a point configuration is spread over axis-aligned ranges; CNF hardness for DPLL is widely conjectured to track ‘uniformity’ of clause structure (Tseitin on expanders and random $\alpha = 4.267$ formulas both exhibit balanced literal distributions and yield exponentially large DPLL trees). The 2-marginal box discrepancy is the simplest non-trivial Tusnady-style geometric discrepancy that ignores 1-marginal polarity bias (already known to be a weak heuristic predictor for DPLL via Jeroslow-Wang) but captures the next-order clause-pair polarity correlations relevant to unit-propagation cascades. The bridge proposes ‘spread’ of the clause-polarity cloud as a structural CNF invariant within the Chazelle-Matousek-Bourgain dispersion programme, distinct from spectral, additive-combinatorial, and curvature-based attempts.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0449e10b78ef05b0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 150 accepted unsat 3-CNFs (30 seeds $\times$ $n \in \{12, 14, 16, 18, 20\}$, $\alpha = 4.5$, DPLL cap 10^6), compute Spearman ρ between $-D_2(F)/\sqrt{m}$ and $\log_2 t^*(F)$. SUPPORTED iff $\rho \geq 0.30$ AND two-sided bootstrap p (1000 resamples) < 0.05 ; FALSIFIED otherwise.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Tusnady discrepancy CNF refutation DPLL tree resolution-axis-aligned box discrepancy clause polarity proof complexity-geometric discrepancy random 3-SAT DPLL tree size lower bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import json
from collections import defaultdict
from itertools import combinations

def generate_3cnf(n, m, seed):
    random.seed(seed)
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause_vars = random.sample(variables, 3)
        clause = []
        for var in clause_vars:
            sign = random.choice([-1, 1])
            clause.append((var, sign))
        clauses.append(clause)
    return clauses

def is_unsatisfiable(clauses, n):
    def dpll(clauses, assignment):
        nonlocal nodes
        nodes += 1
        if nodes > 10**6:
            return None

        if not clauses:
            return True

        for clause in clauses:
            satisfied = False
            for lit in clause:
                var, sign = lit
                if var in assignment and assignment[var] == sign:
                    satisfied = True
                    break
            if not satisfied:
                return False

        for var in range(1, n + 1):
            if var not in assignment:
                for sign in [-1, 1]:
                    new_assignment = assignment.copy()
                    new_assignment[var] = sign
                    if dpll(clauses, new_assignment):
                        return True
                return False
        return False

    nodes = 0
    return dpll(clauses, {}) is False

def compute_d2(clauses, n, m):
    c_ij_st = defaultdict(int)
```

```

for clause in clauses:
    for (var1, sign1), (var2, sign2) in combinations(clause, 2):
        if var1 < var2:
            c_ij_st[(var1, var2, sign1, sign2)] += 1
        else:
            c_ij_st[(var2, var1, sign2, sign1)] += 1

mu_ij_st = m * (3 / n) * (2 / (n - 1)) * (1 / 4)
max_diff = 0
for (i, j, s, t) in c_ij_st:
    diff = abs(c_ij_st[(i, j, s, t)] - mu_ij_st)
    if diff > max_diff:
        max_diff = diff
return max_diff

def spearman_rank_correlation(x, y):
    n = len(x)
    if n == 0:
        return 0.0
    rank_x = sorted(range(n), key=lambda i: x[i])
    rank_y = sorted(range(n), key=lambda i: y[i])
    d_squared = sum((rank_x[i] - rank_y[i])**2 for i in range(n))
    return 1 - (6 * d_squared) / (n * (n**2 - 1))

def run_trial(seed):
    n_values = [12, 14, 16, 18, 20]
    alpha = 4.5
    instances = []
    for n in n_values:
        m = int(alpha * n)
        attempts = 0
        while attempts < 100:
            clauses = generate_3cnf(n, m, seed + attempts)
            if is_unsatisfiable(clauses, n):
                d2 = compute_d2(clauses, n, m)
                instances.append((n, m, d2))
                break
            attempts += 1

    if len(instances) < len(n_values):
        return {
            "metric_name": "spearman_correlation",
            "metric_value": 0.0,
            "instances_tested": len(instances),
            "conjecture_holds": False,
            "counterexample": "failed_to_generate_unsat_instances"
        }

    x = [-d2 / math.sqrt(m) for (n, m, d2) in instances]
    y = [math.log2(m) for (n, m, d2) in instances]
    correlation = spearman_rank_correlation(x, y)

    return {
        "metric_name": "spearman_correlation",
        "metric_value": correlation,
        "instances_tested": len(instances),

```

```

        "conjecture_holds": correlation >= 0.30,
        "counterexample": "" if correlation >= 0.30 else f"correlation={correlation}"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [random.randint(1, 1000)]
    trials = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {json.dumps({'seed': seed, **trial})}")
        trials.append(trial)

    metric_values = [trial["metric_value"] for trial in trials]
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean)**2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for trial in trials if trial["conjecture_holds"]) / len(trials)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((seeds[i] for i, trial in enumerate(trials) if not trial["conjecture_holds"]), None)
        if first_failing_seed is not None:
            print(f"RESULT: FALSIFIED counterexample=\"{trials[seeds.index(first_failing_seed)]['counterexample']}\"")
        else:
            print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

Seed	Metric value	Holds?	Counterexample
11	0.9	☐	
23	0.5	☐	
37	0.19999999999999996	☐	correlation=0.19999999999999996
53	1.0	☐	
71	-0.5	☐	correlation=-0.5
89	0.7	☐	
103	1.0	☐	
127	0.19999999999999996	☐	correlation=0.19999999999999996
149	0.9	☐	
167	0.5	☐	
191	0.7	☐	
211	1.0	☐	
233	0.19999999999999996	☐	correlation=0.19999999999999996
257	0.9	☐	
277	0.9	☐	
311	0.4	☐	
347	0.0	☐	correlation=0.0
389	0.09999999999999998	☐	correlation=0.09999999999999998
421	0.19999999999999996	☐	correlation=0.19999999999999996
463	0.9	☐	
503	0.9	☐	
547	0.7	☐	
593	1.0	☐	
631	0.4	☐	
677	1.0	☐	
727	1.0	☐	

Seed	Metric value	Holds?	Counterexample
773	1.0	□	correlation=-0.5
821	0.7	□	
877	-0.5	□	
929	0.9	□	

Aggregate statistics:

Statistic	Value
n_seeds	30
metric_mean	0.5933333333333334
metric_std	0.43781064505050604
metric_ci95_half	0.1598658441400331
metric_min	-0.5
metric_max	1.0
support_fraction	0.7333333333333333

7. Test stdout (last 2KB)

```
.09999999999999998"}
TRIAL: {"seed": 421, "metric_name": "spearman_correlation", "metric_value": 0.19999999999999996, "instances_tested": 5}
TRIAL: {"seed": 463, "metric_name": "spearman_correlation", "metric_value": 0.9, "instances_tested": 5}
TRIAL: {"seed": 503, "metric_name": "spearman_correlation", "metric_value": 0.9, "instances_tested": 5}
TRIAL: {"seed": 547, "metric_name": "spearman_correlation", "metric_value": 0.7, "instances_tested": 5}
TRIAL: {"seed": 593, "metric_name": "spearman_correlation", "metric_value": 1.0, "instances_tested": 5}
TRIAL: {"seed": 631, "metric_name": "spearman_correlation", "metric_value": 0.4, "instances_tested": 5}
TRIAL: {"seed": 677, "metric_name": "spearman_correlation", "metric_value": 1.0, "instances_tested": 5}
TRIAL: {"seed": 727, "metric_name": "spearman_correlation", "metric_value": 1.0, "instances_tested": 5}
TRIAL: {"seed": 773, "metric_name": "spearman_correlation", "metric_value": 1.0, "instances_tested": 5}
TRIAL: {"seed": 821, "metric_name": "spearman_correlation", "metric_value": 0.7, "instances_tested": 5}
TRIAL: {"seed": 877, "metric_name": "spearman_correlation", "metric_value": -0.5, "instances_tested": 5}
TRIAL: {"seed": 929, "metric_name": "spearman_correlation", "metric_value": 0.9, "instances_tested": 5}
RESULT: FALSIFIED counterexample="correlation=0.19999999999999996" first_failing_seed=37
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

Metric definition bug + n-too-small: the statement specifies a Spearman correlation across all 150 instances (30 seeds × 5 sizes) with $p < 0.05$, but the per-seed metric is computed on only 5 points (one per n), where Spearman values are discretized to $\{-1, -0.9, -0.7, \dots\}$ and two-sided $p < 0.05$ requires $|\rho| = 1.0$ exactly. The $p < 0.05$ gate has effectively been dropped, and ‘holds’ reduces to $\text{metric} \geq 0.3$ on $n=5$. Worse, the correlation is dominated by the n-trend ($\log t^*(F)$ grows exponentially with n

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test’s RESULT line reports FALSIFIED with counterexample correlation=0.20 at seed=37, and the critic flags that the pre-registered 150-instance Sp | next: Recompute Spearman ρ on the pooled 150-instance set (not per-seed n=5 subsets), partial out the n-trend by within-n ranking or residualizing $\log_2 t^*(F)$ on n before correlating with $-D_2/\sqrt{m}$.

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	255011
2	preregistration	claude_max	opus	0	0	6167
3	novelty	claude_max	opus	0	0	4251
4	novelty	claude_max	opus	0	0	9352
5	test_gen	mistral	codestral-latest	0	0	321482
6	test_gen	mistral	codestral-latest	0	0	319082
7	critic	claude_max	opus	0	0	17561
8	judge	claude_max	opus	0	0	17086

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 949994 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 2}

(full prompt+response transcripts available in `research/audit/d8440effcf4b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d8440effcf4b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d8440effcf4b.tar.gz` (if generated)